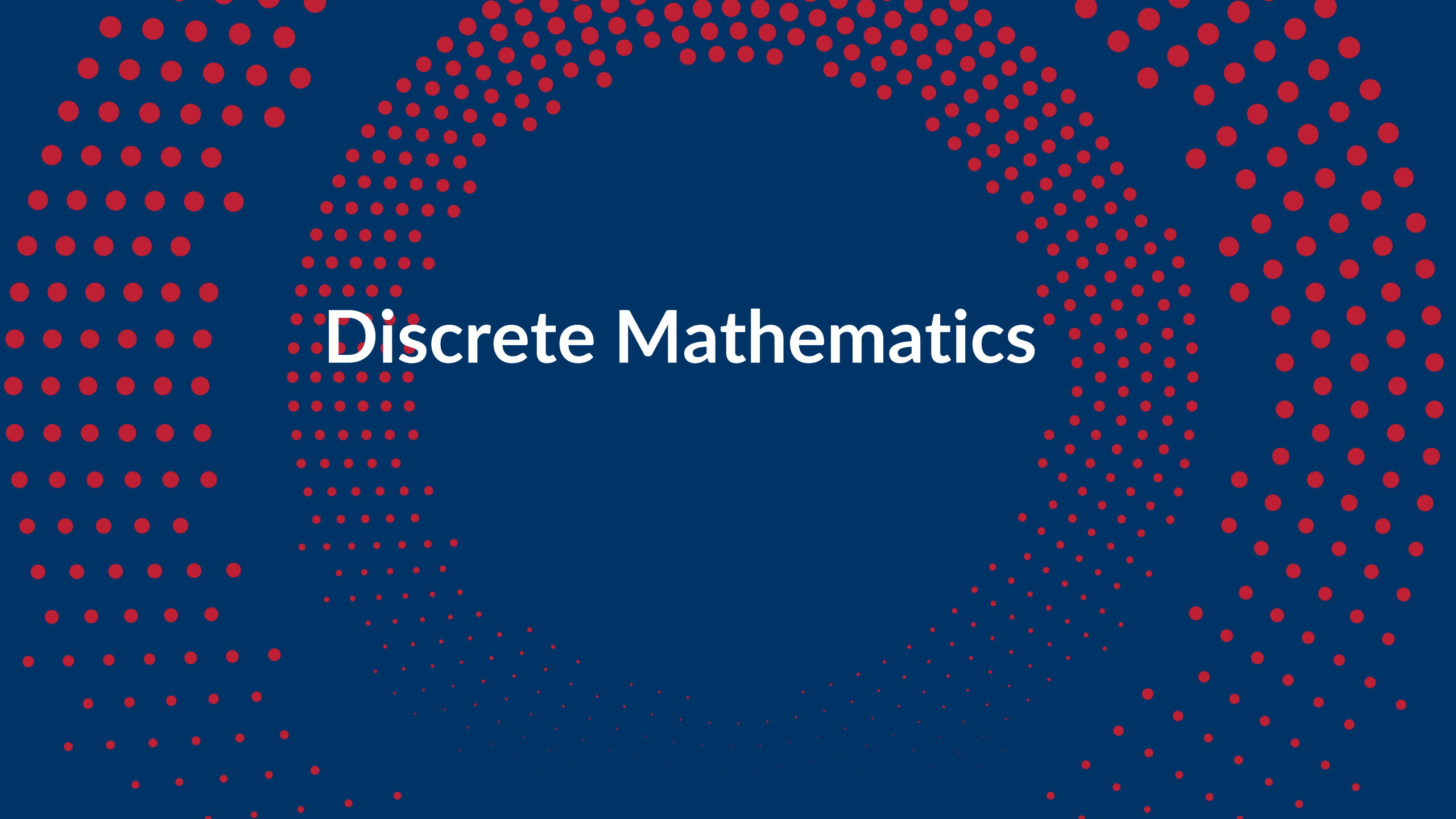




HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



Discrete Mathematics



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Discrete Mathematics

ONE LOVE. ONE FUTURE.

PART 1

COMBINATORIAL THEORY

(Lý thuyết tổ hợp)

PART 2

GRAPH THEORY

(Lý thuyết đồ thị)

CONTENT OF PART 2

Chapter 1. Fundamental concepts

Chapter 2. Graph representation

Chapter 3. Graph Traversal

Chapter 4. Tree and Spanning tree

Chapter 5. Shortest path problem

Chapter 6. Maximum flow problem



GRAPH TRAVERSAL (GRAPH SEARCHING)

Searching a graph means systematically following the edges of the graph so as to visit the vertices.

2 algorithms:

- Breadth First Search – BFS
- Depth First Search – DFS

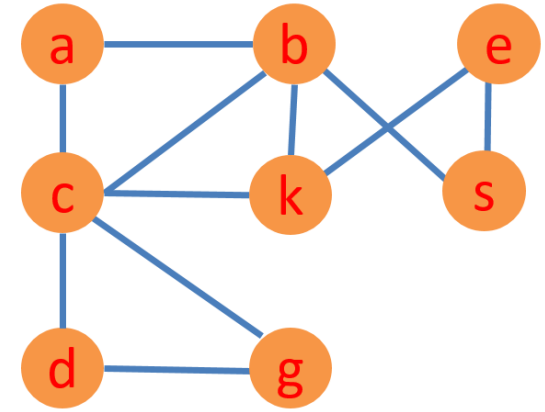
- **Breadth First Search**
- Depth First Search
- Applications

BREADTH FIRST SEARCH

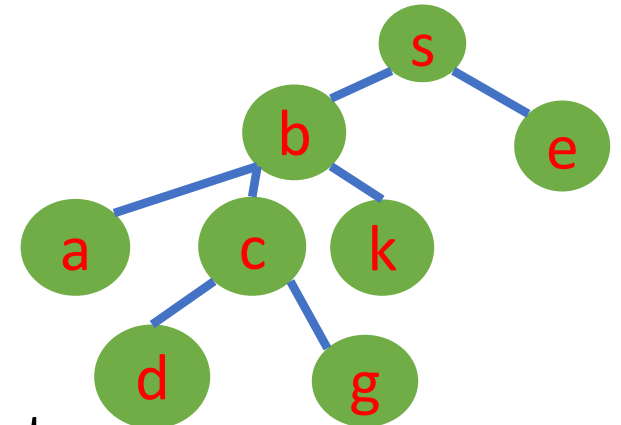
- Given
 - a graph $G=(V,E)$ – set of vertices and edges
 - a distinguished source vertex s
- Breadth first search systematically explores the edges of G to discover every vertex that is reachable from s .
- It also produces a ‘breadth first tree’ with root s that contains all the vertices reachable from s .
- For any vertex v reachable from s , the path in the breadth first tree corresponds to the shortest path in graph G from s to v .
- It works on both directed and undirected graphs.

BREADTH FIRST SEARCH

- Given
 - a graph $G=(V,E)$ – set of vertices and edges
 - a distinguished source vertex s
- Breadth first search systematically explores the edges of G to discover every vertex that is reachable from s .
- For any vertex v reachable from s , the path in the breadth first tree corresponds to the shortest path in graph G from s to v .



$G=(V, E)$



BFS(s) tree

BFS creates a **BFS tree** containing s as the root and all vertices that is reachable from s

BREADTH FIRST SEARCH

BFS(s)

// Breadth first search starts from vertex s

visited[s] $\leftarrow$ 1; //visited

$Q \leftarrow \emptyset$; enqueue(Q, s); // insert s into Q

while ($Q \neq \emptyset$)

{

$u \leftarrow$ dequeue(Q); // Remove u from Q

 for $v \in \text{Adj}[u]$

 if (visited[v] == 0) //not visited yet

 {

 visited[v] $\leftarrow$ 1; //visited

 enqueue(Q, v) // insert v into Q

 }

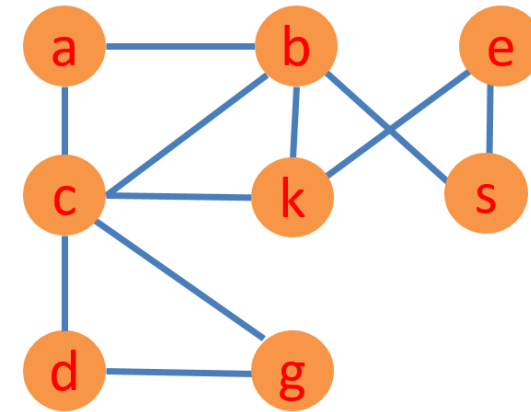
 }

(*Main Program*)

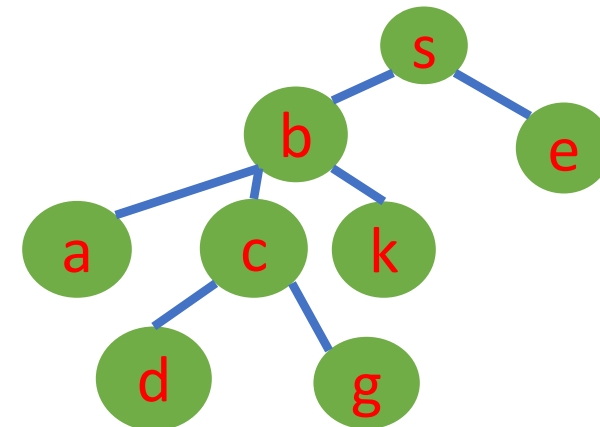
main ()

 for $s \in V$ // Initialize
 visited[s] $\leftarrow$ 0;

 for $s \in V$
 if (visited[s]==0) BFS(s);



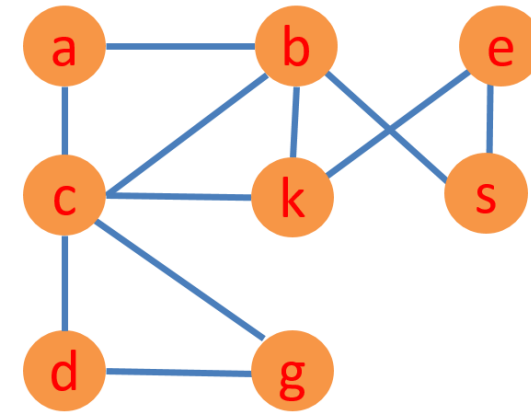
$G=(V, E)$



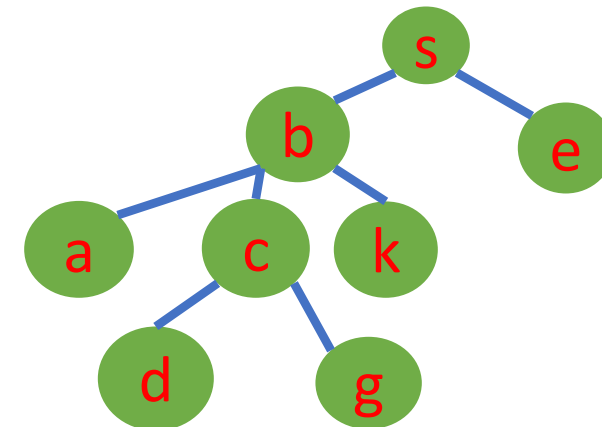
BFS(s) tree

BREADTH FIRST SEARCH

- From s: can go to b and e. Visit them and insert them into queue: $Q = \{b, e\}$
- Dequeue (Q): remove b out of Q, then $Q = \{e\}$
 - From b: can go to a, c, k, s. But s was visited, so we visit only a, c, k; and insert them into queue: $Q = \{e, a, c, k\}$
- Dequeue(Q): remove e out of Q, then $Q = \{a, c, k\}$
 - From e: can go to k, s. But all of them were visited.
- Dequeue(Q): remove a out of Q, then $Q = \{c, k\}$
 - From a: can go to b, c. But these vertices were all visited.
- Dequeue(Q): remove c out of Q, then $Q = \{k\}$
 - From c: can go to a, b, d, g, k. But a, b, k were visited, so we visit only d, g; and insert them into queue: $Q = \{k, d, g\}$
- Dequeue(Q): remove k out of Q, then $Q = \{d, g\}$
 - From k: can go to b, c, e. But these vertices were all visited.
- Dequeue (Q): remove d from Q, then $Q = \{g\}$
 - From d: can go to c, g. But these vertices were all visited.
- Dequeue(Q): remove g from Q, then $Q = \text{empty}$
 - From g: can go to d, c. But these vertices were all visited.
- Q is now empty. All vertices of the graph were visited. Algorithm is finished



$G=(V, E)$



BFS(s) tree

COMPUTATION TIME OF BFS

```
BFS(s)
// Breadth first search starts from vertex s
visited[s] ← 1; //visited
Q ← ∅; enqueue(Q,s); // insert s into Q
while (Q ≠ ∅)
{
    u ← dequeue(Q); // Remove u from Q
    for v ∈ Adj[u]
        if (visited[v] == 0) //not visited yet
        {
            visited[v] ← 1; //visited
            enqueue(Q,v) // insert v into Q
        }
}
```

(*Main Program*)

```
main ()
for s ∈ V // Initialize
    visited[s] ← 0;

for s ∈ V
    if (visited[s]==0) BFS(s);
```

- Initialize: need $O(|V|)$.
- The loop for
 - Each vertex is inserted into and removed from queue exactly once, each operation needs $O(1)$. So, the total computation time with queue is $O(|V|)$.
 - The adjacency list of each vertex is traversed exactly once. The total length of all adjacency list is $O(|E|)$.
- In total, the computation time of BFS(s) is $O(|V|+|E|)$, linear to the size of adjacency list that represents the graph.

BREADTH FIRST SEARCH

- **Input:** Graph $G = (V, E)$, either undirected or directed graph.
vertex $s \in V$: source vertex
- **Output:**
 - $d[v]$ = distance (length of shortest path) from s to v , where $v \in V$. $d[v] = \infty$ if v is not reachable from s .
 - $pred[v] = u$ the vertex that is preceding v in the path from s to v with length of $d[v]$.
 - Build BFS tree consisting of root s and all vertices that are reachable from s .

BREADTH FIRST SEARCH

```
BFS(s)
//Breadth first search starts from vertex s
visited[s] ← 1; //visited
d[s] ← 0; pred[s] ← NULL;
Q ← ∅; enqueue(Q,s); //Insert s into Q
while (Q ≠ ∅)
{
    u ← dequeue(Q); //Remove u from Q
    for v ∈ Adj[u]
        if (visited[v] == 0) //not visited
        yet
        {
            visited[v] ← 1; //visited
            d[v] ← d[u] + 1; pred[v] ← u;
            enqueue(Q,v) //Insert v into
            Q
        }
    }
}

(*Main Program*)
for s ∈ V //initialize
{
    visited[s] ← 0; d[s] ← ∞; pred[s] ←
    NULL;
}
for s ∈ V
    if (visited[s]==0) BFS(s);
```

Q: queue contain all visited vertices
visited[v]: mark visit state of vertex v
d[v]: distance from to v
pred[v]: vertex that is preceding v

- Breadth First Search
- **Depth First Search**
- Applications

DEPTH FIRST SEARCH (DFS)

- It searches 'deeper' the graph when possible.
- Starts at the selected node and explores as far as possible along each branch before backtracking.

(* Main Program*)

```
1. for  $s \in V$ 
2.     visited[s]  $\leftarrow$  false
3. for  $s \in V$ 
4.     if (visited[s] == false)
5.         DFS(s)
```

DFS(s)

```
1. visited[s]  $\leftarrow$  true // Visist s
2. for each  $v \in Adj[s]$ 
3.     if (visited[v] ==
4.         false)
5.             DFS(v)
```

COMPUTATION TIME OF DFS

```
(* Main Program*)
1.  for  $s \in V$ 
2.      visited[s]  $\leftarrow$  false
3.  for  $s \in V$ 
4.      if (visited[s] == false)
5.          DFS(s)
```

```
DFS(s)
1.  visited[s]  $\leftarrow$  true // Visit s
2.  for each  $v \in \text{Adj}[s]$ 
3.      if (visited[v] == false)
4.          DFS(v)
```

- In main: the loops for on lines 1-2 and 3-5 require $O(|V|)$, excluding computation time of statement DFS(s).
 - In DFS(s): the loop for on line 2 performs to traverse edges of graph:
 - Lines 3-4 of DFS(s) perform $|\text{Adj}[s]|$ times
 - Each edge is traversed exactly once if graph is directed, otherwise exactly twice if graph is undirected
- The total computation time of DFS(s) in the main program is $\sum_{s \in V} |\text{Adj}[s]| = O(|E|)$
- Thus, computation time of DFS is $O(|V| + |E|)$.

DFS: PATH FROM VERTEX s TO ALL OTHER VERTICES OF GRAPH

(* Main Program*)

```
1. for  $s \in V$ 
2.     visited[s]  $\leftarrow$  false
3. for  $s \in V$ 
4.     if (visited[s] == false)
5.         DFS(s)
```

(*Main program *)

```
1. for each  $s \in V$ 
2.     pred[s] = NULL;
3.     visited[s] = false;
4. DFS(s);
```

DFS(s)

```
1. visited[s]  $\leftarrow$  true // Visit s
2. for each  $v \in Adj[s]$ 
3.     if (visited[v] == false)
4.         DFS(v)
```

DFS(s)

```
1. visited[s] = true; //Visit s
2. for each  $v \in Adj[s]$ 
3.     if (visited[v] ==
4.         false) {
5.         pred[v]  $\leftarrow$  s;
6.         DFS(v);
7.     }
```

DFS: VISITED ORDER

(*Main program *)

```
1.  for each  $s \in V$ 
2.      pred[s] = NULL;
3.      visited[s] = false;
4.  time = 0
5.  for each  $s \in V$ 
6.      if (visited[s] == false)
        DFS(s);
```

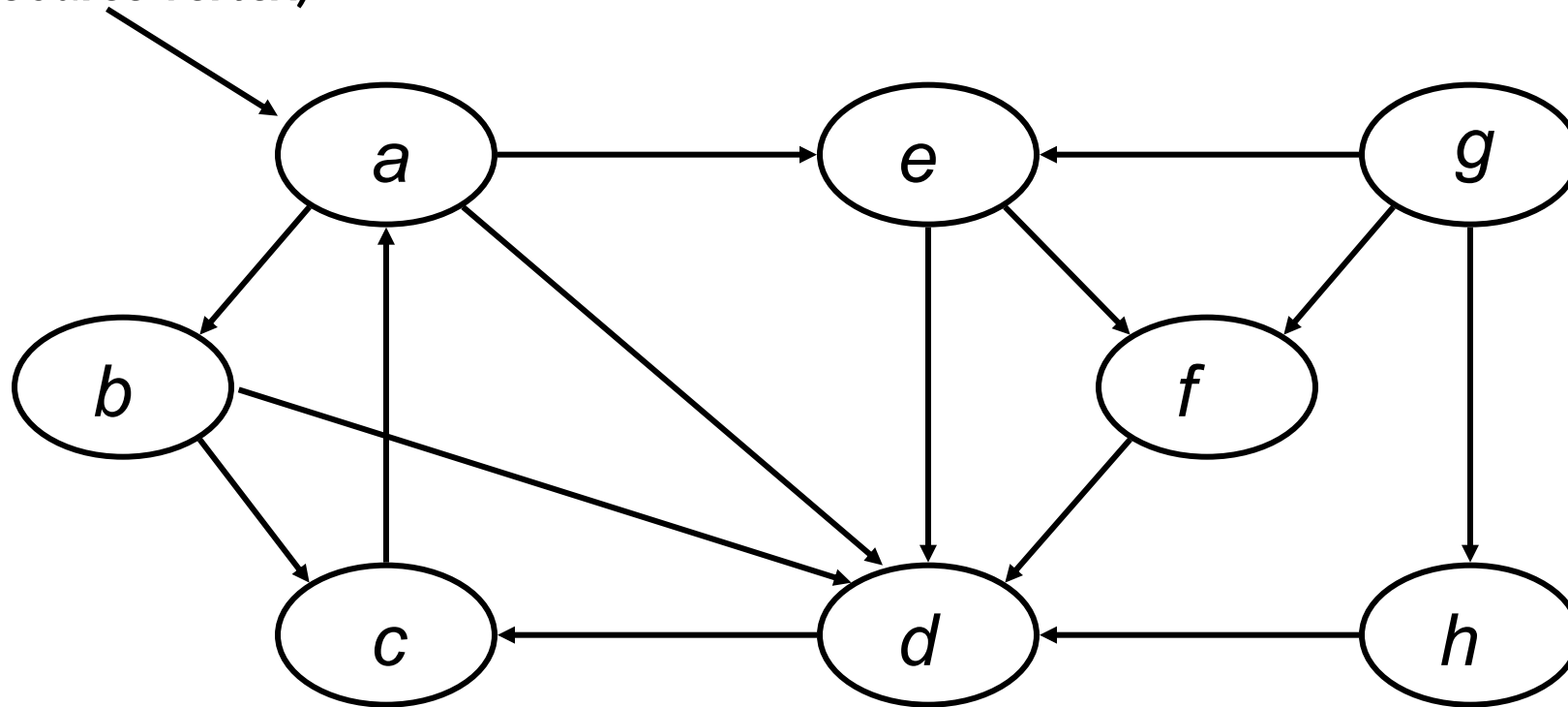
DFS(s)

```
1.  visited[s] = true; //Visit s
2.  time = time + 1
3.  d[s] = time
4.  for each  $v \in Adj[s]$ 
5.      if (visited[v] == false) {
6.          pred[v] ← s;
7.          DFS(v);
8.      }
9.  time = time + 1
10. f[s] = time
```

- Also records timestamps for each vertex
 - $d[v]$ when the vertex v is first discovered
 - $f[v]$ when the vertex v is finished

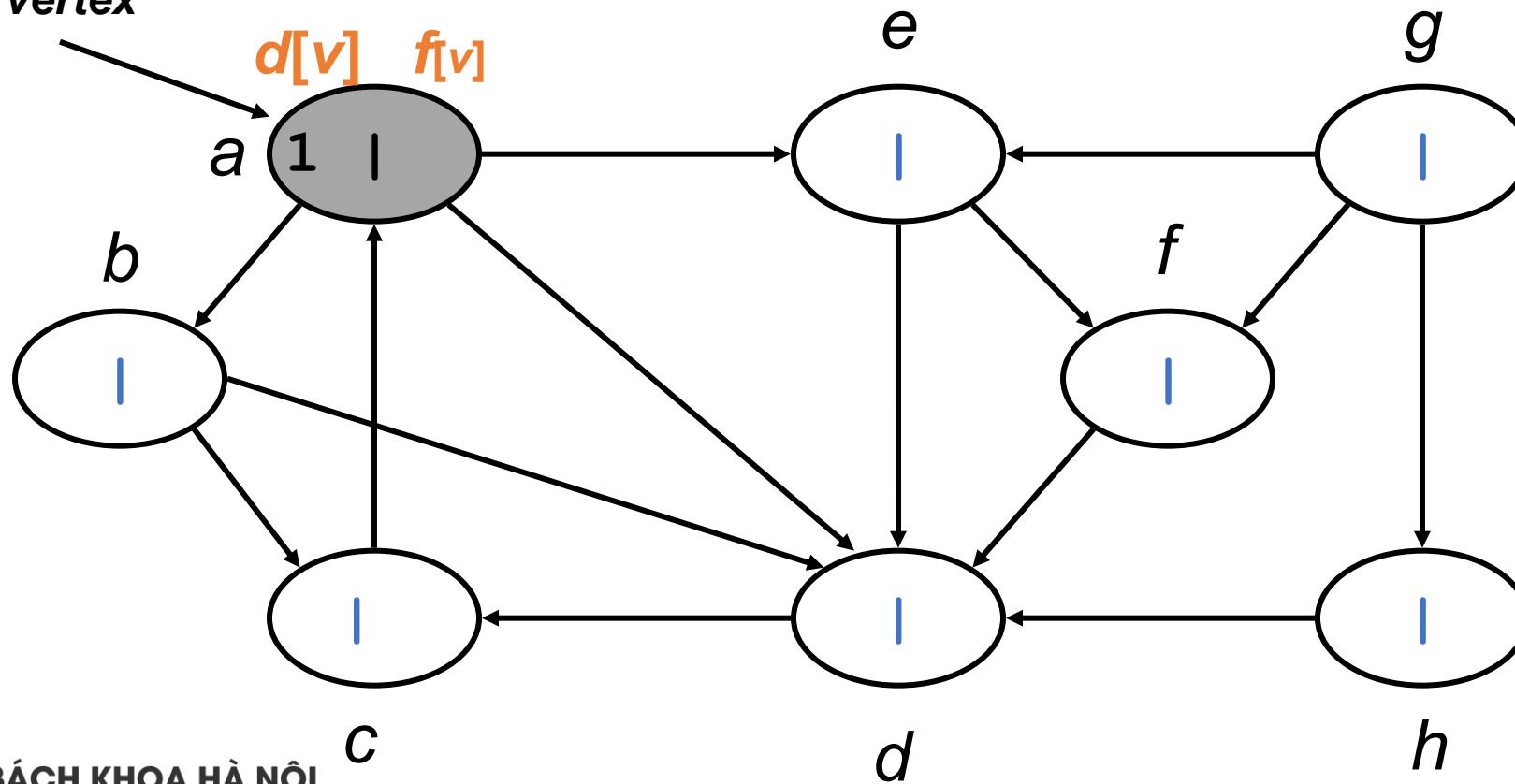
EXAMPLE

Đỉnh xuất phát tìm kiếm
(Source vertex)

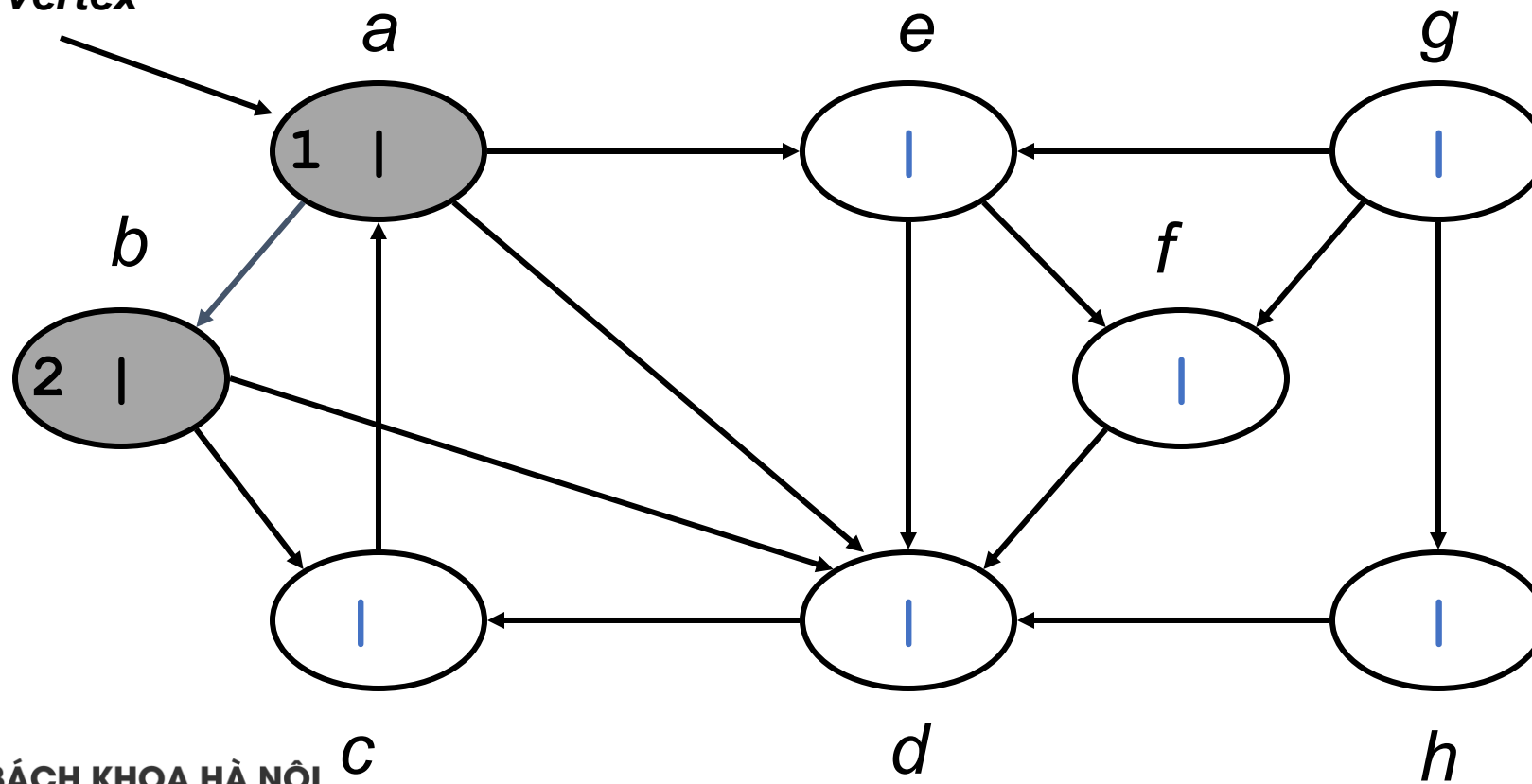


Để hoạt động của thuật toán là xác định, giả thiết rằng ta duyệt các đỉnh trong danh sách kề của một đỉnh theo thứ tự từ điển

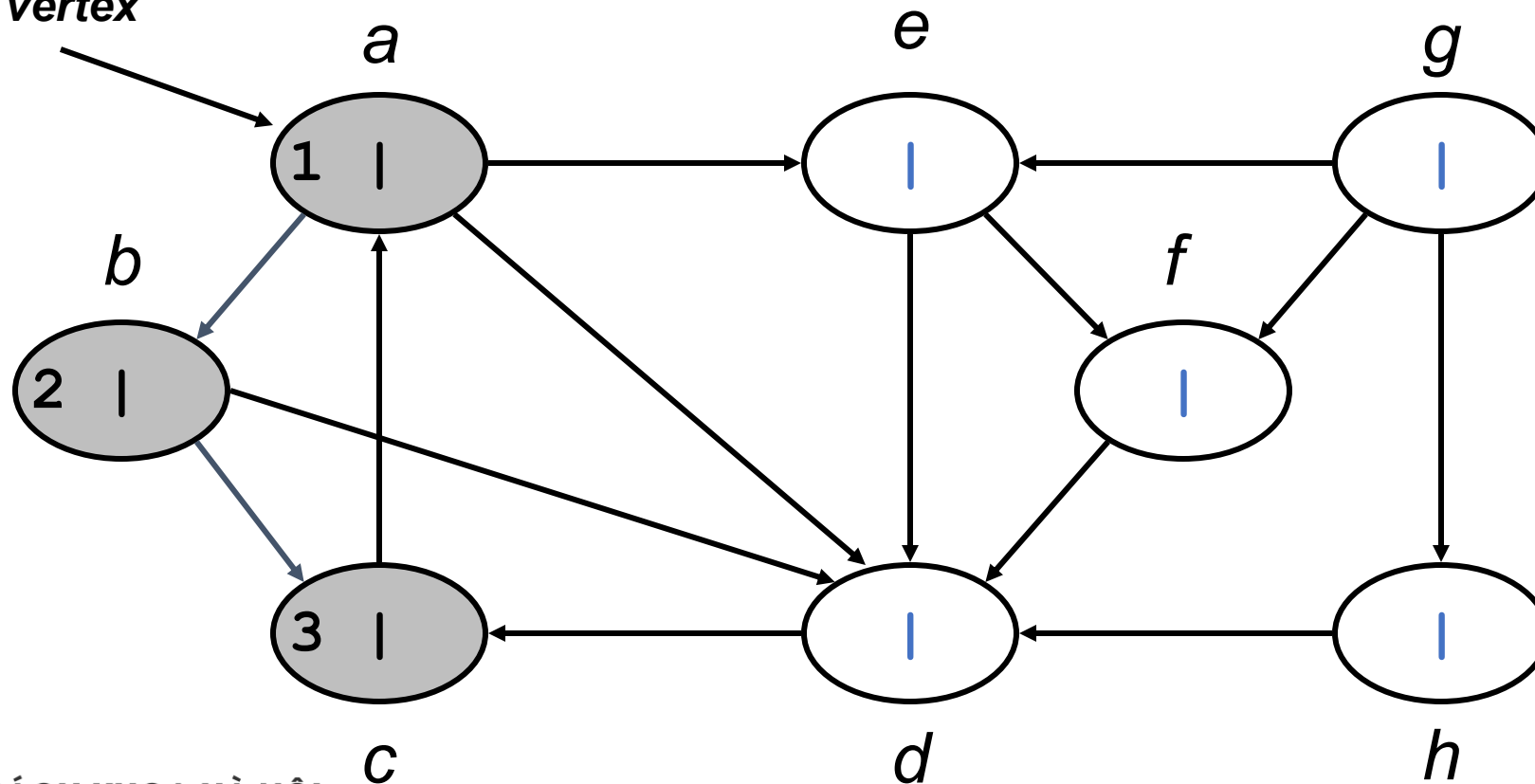
source
vertex



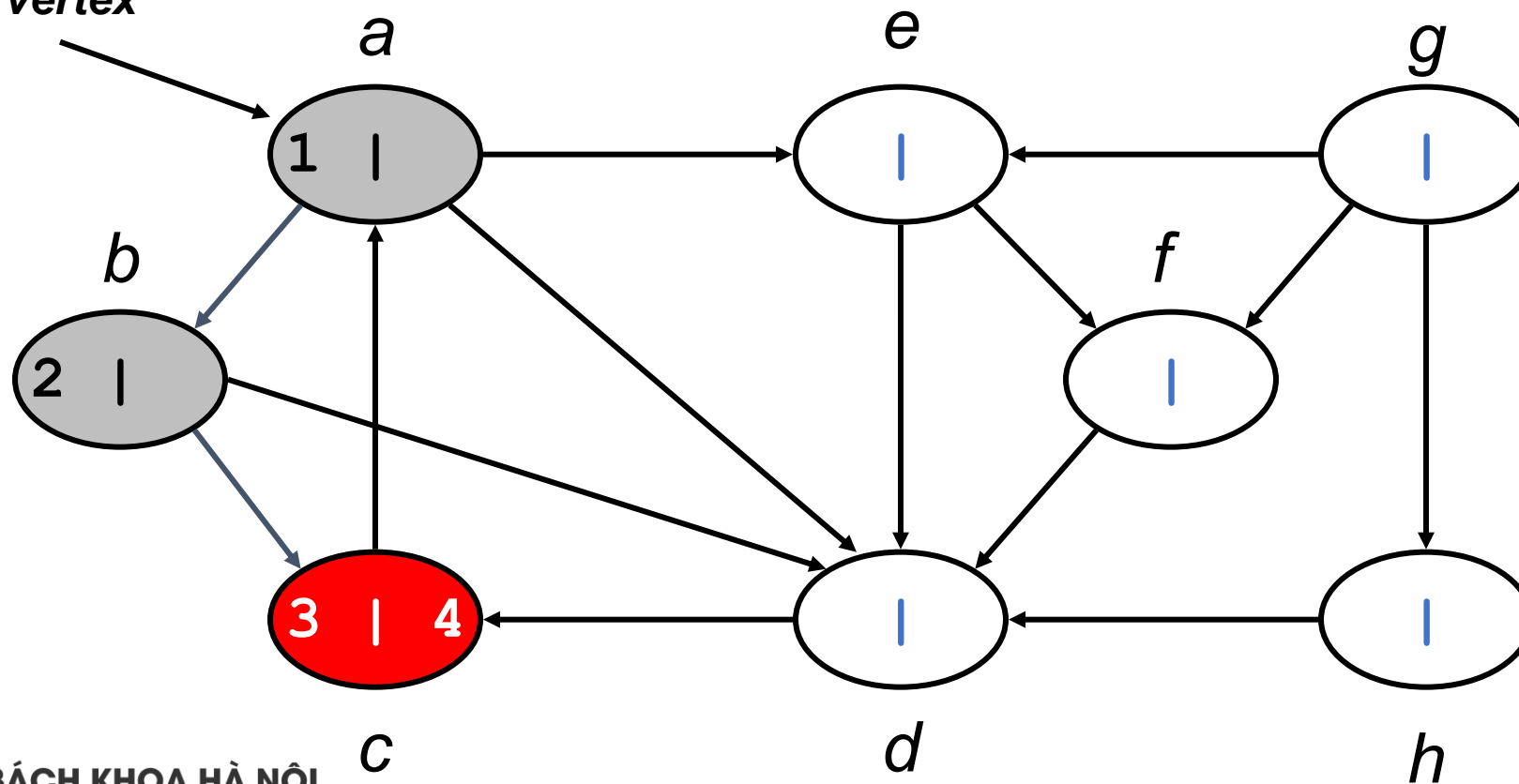
*source
vertex*



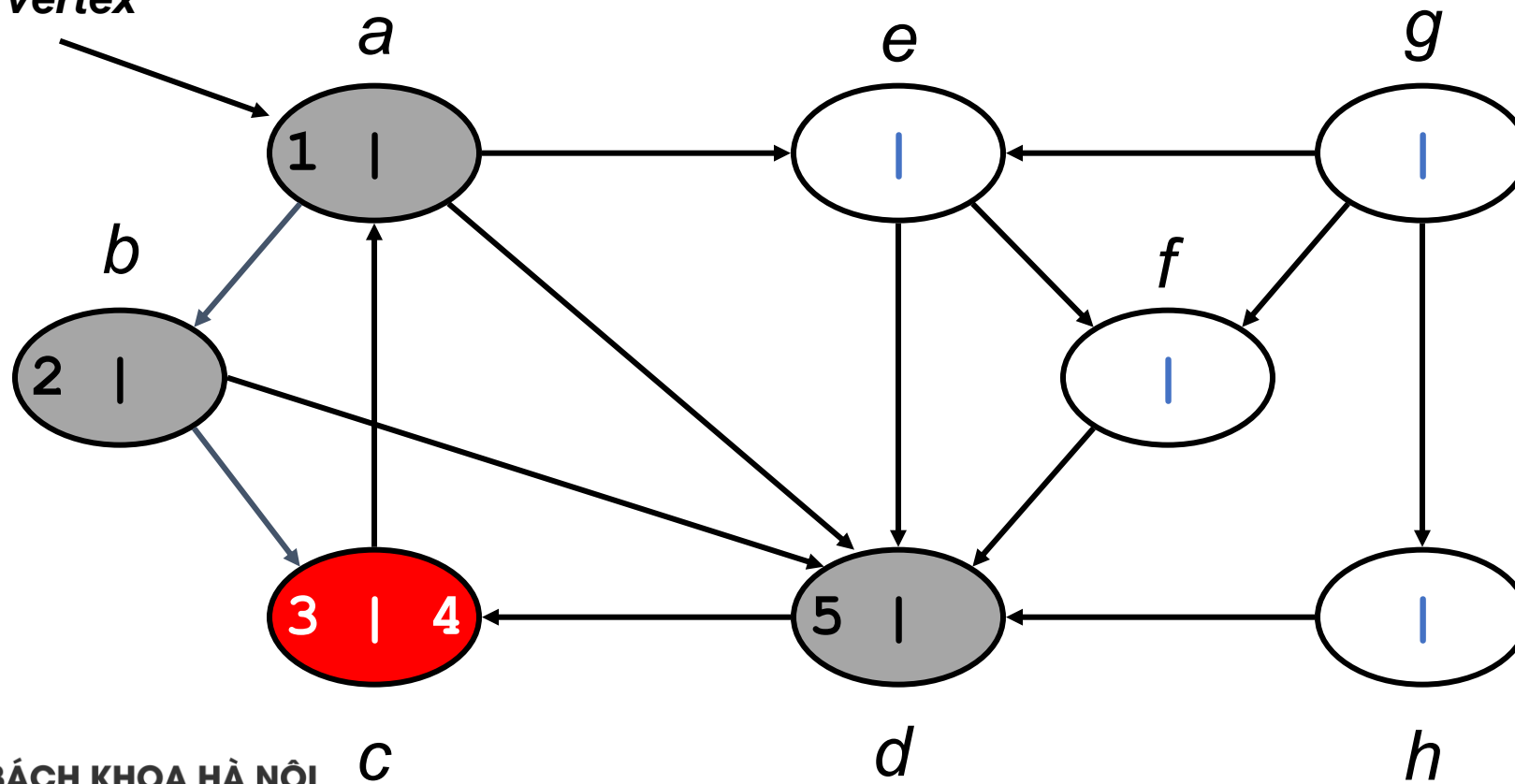
*source
vertex*



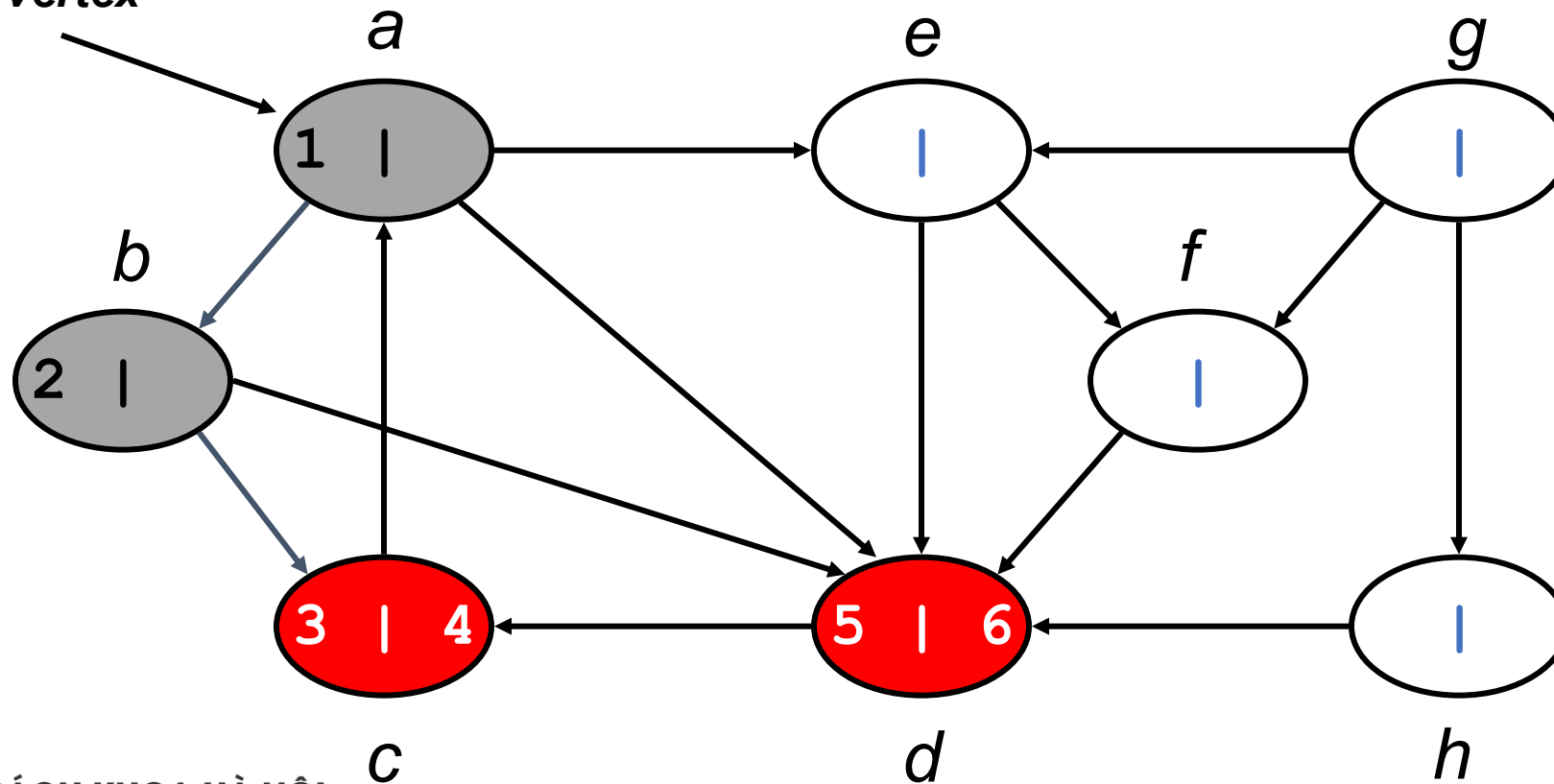
*source
vertex*



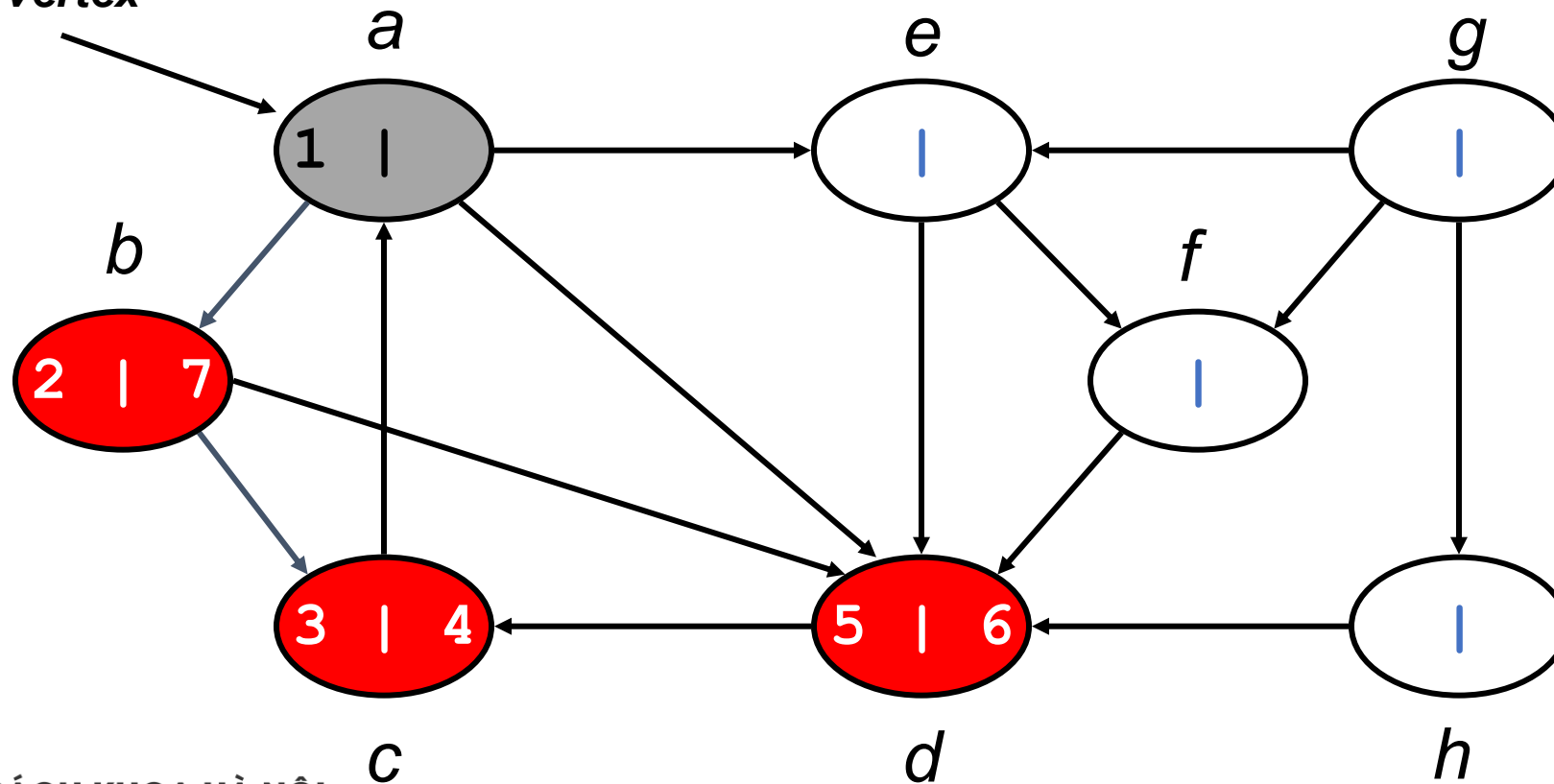
*source
vertex*



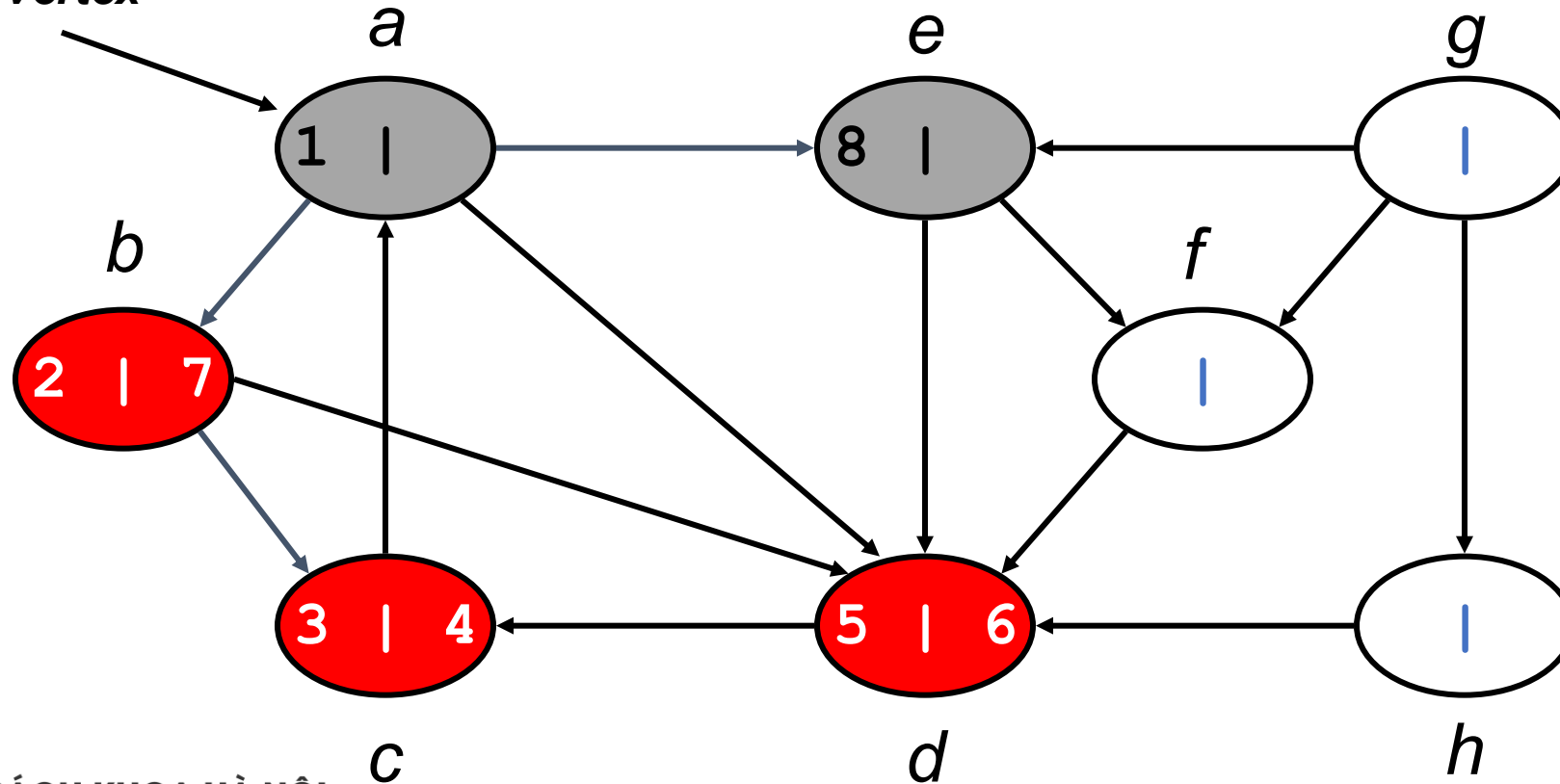
source
vertex



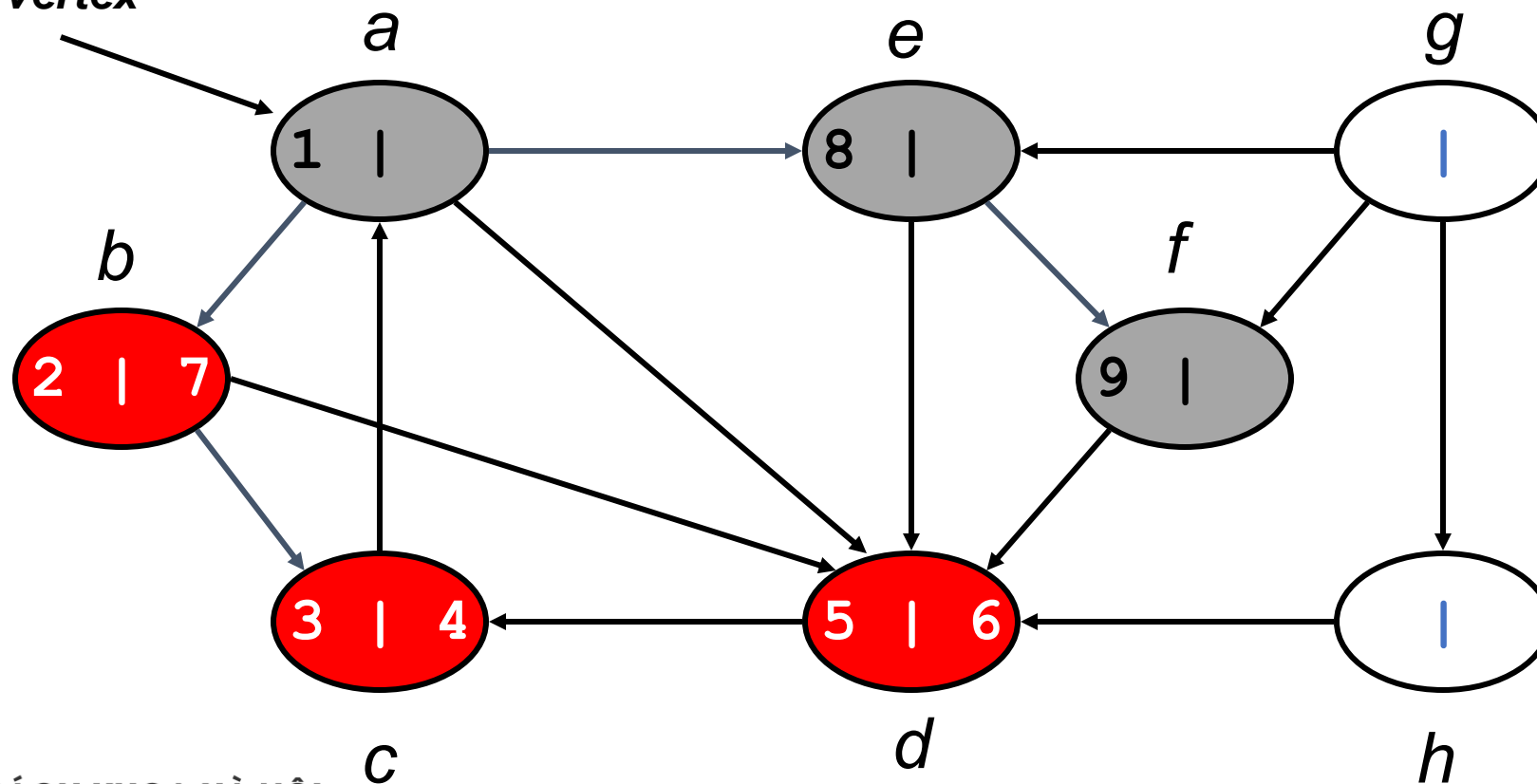
source
vertex



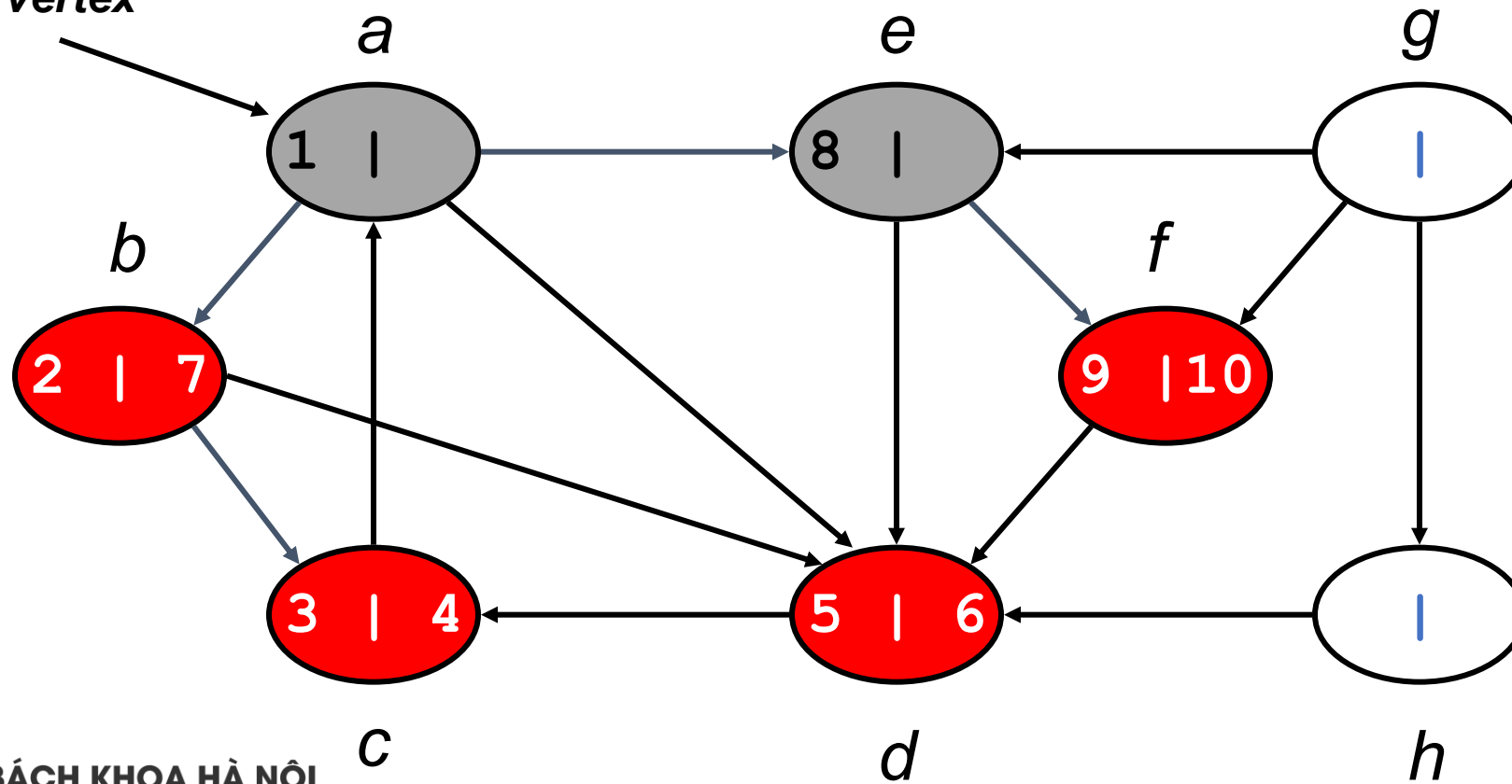
source
vertex



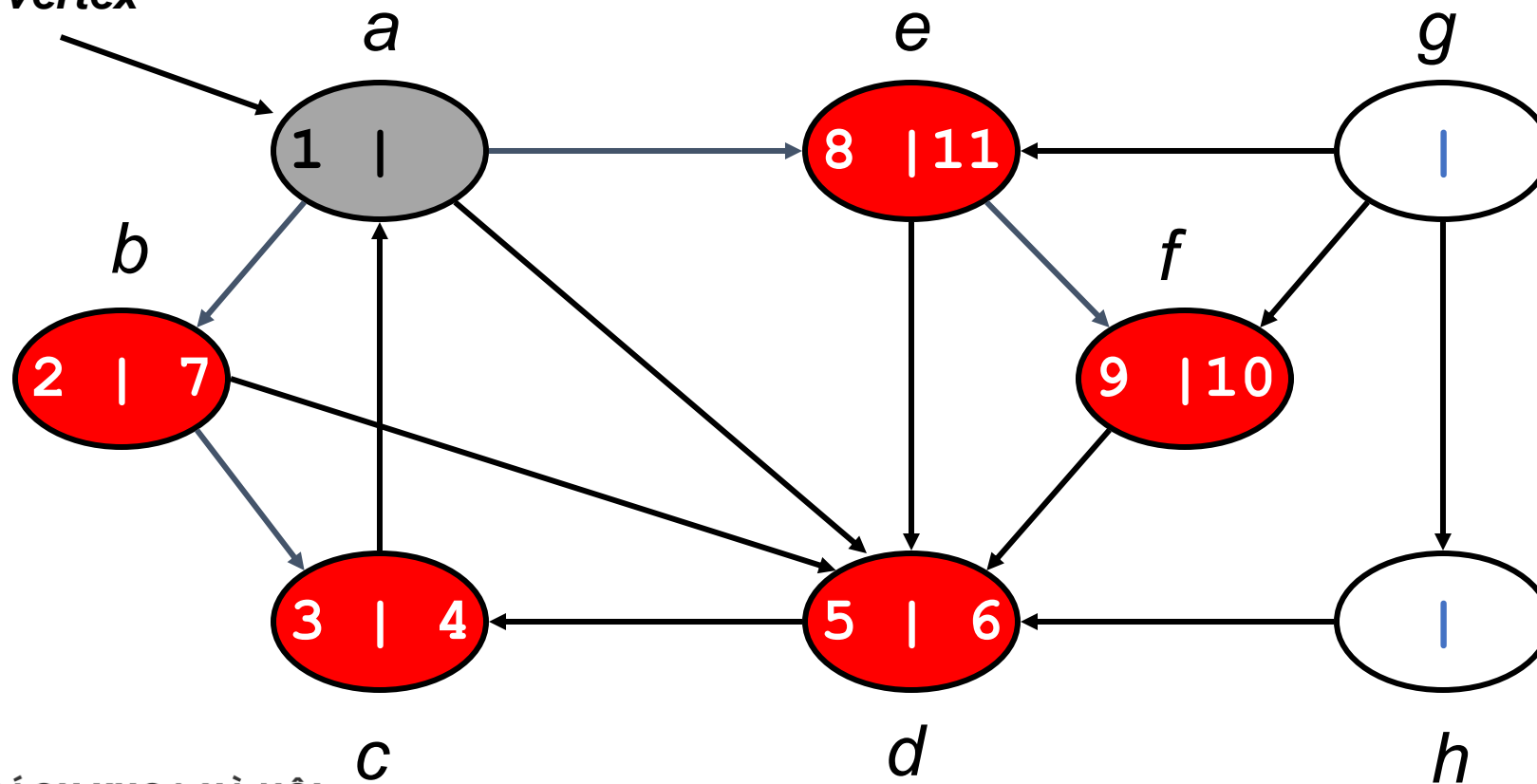
source
vertex



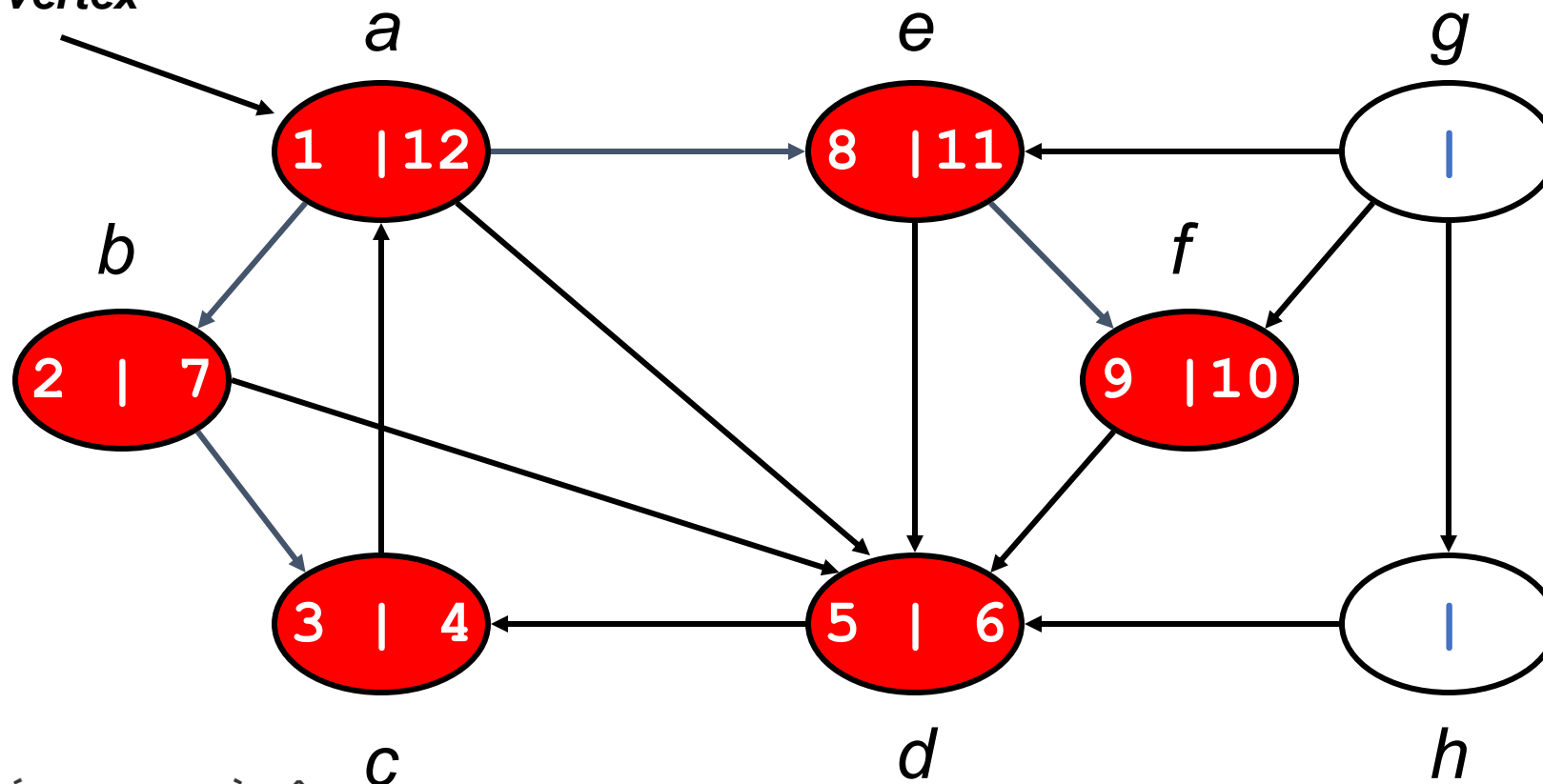
source
vertex



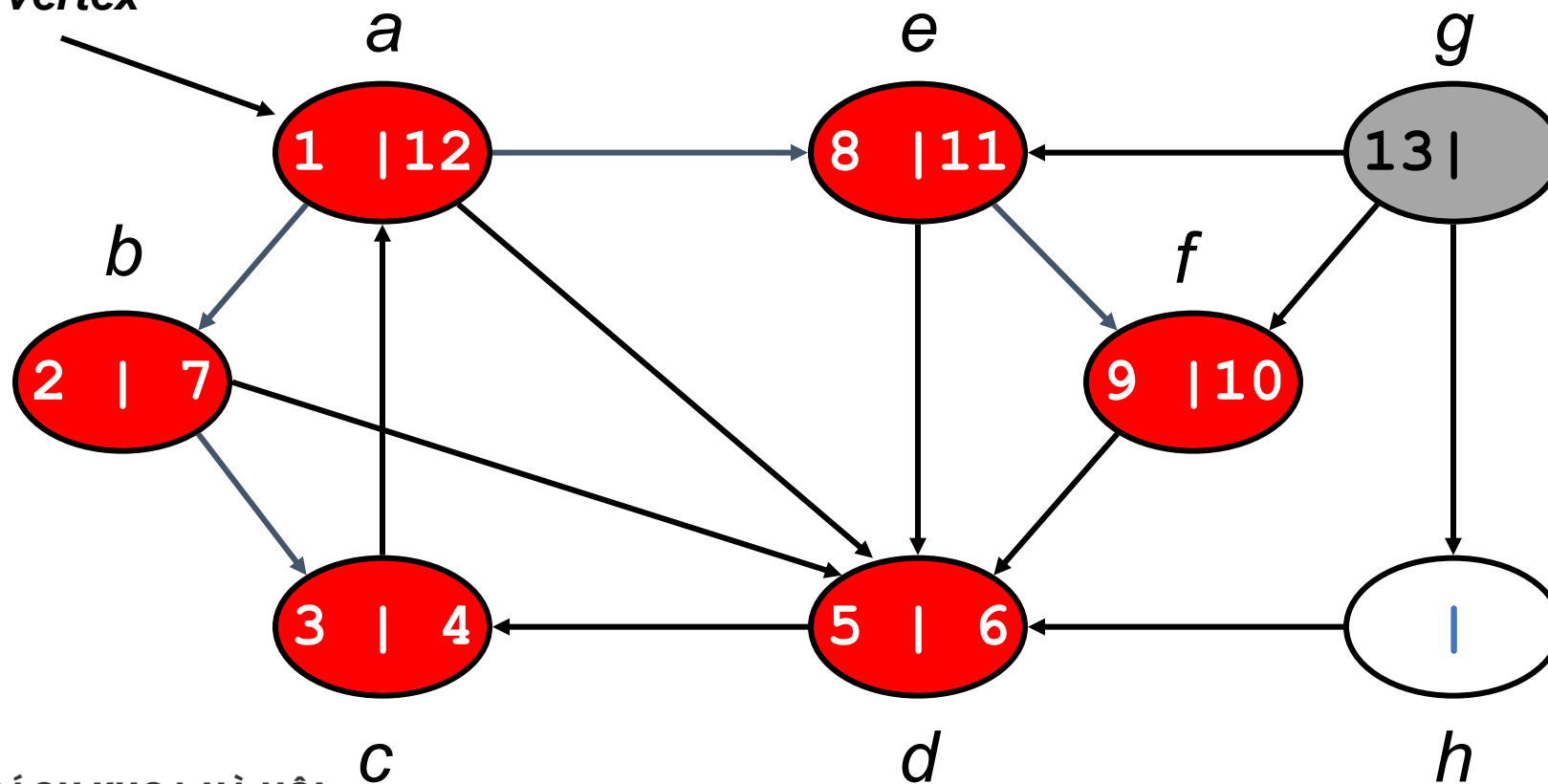
source
vertex



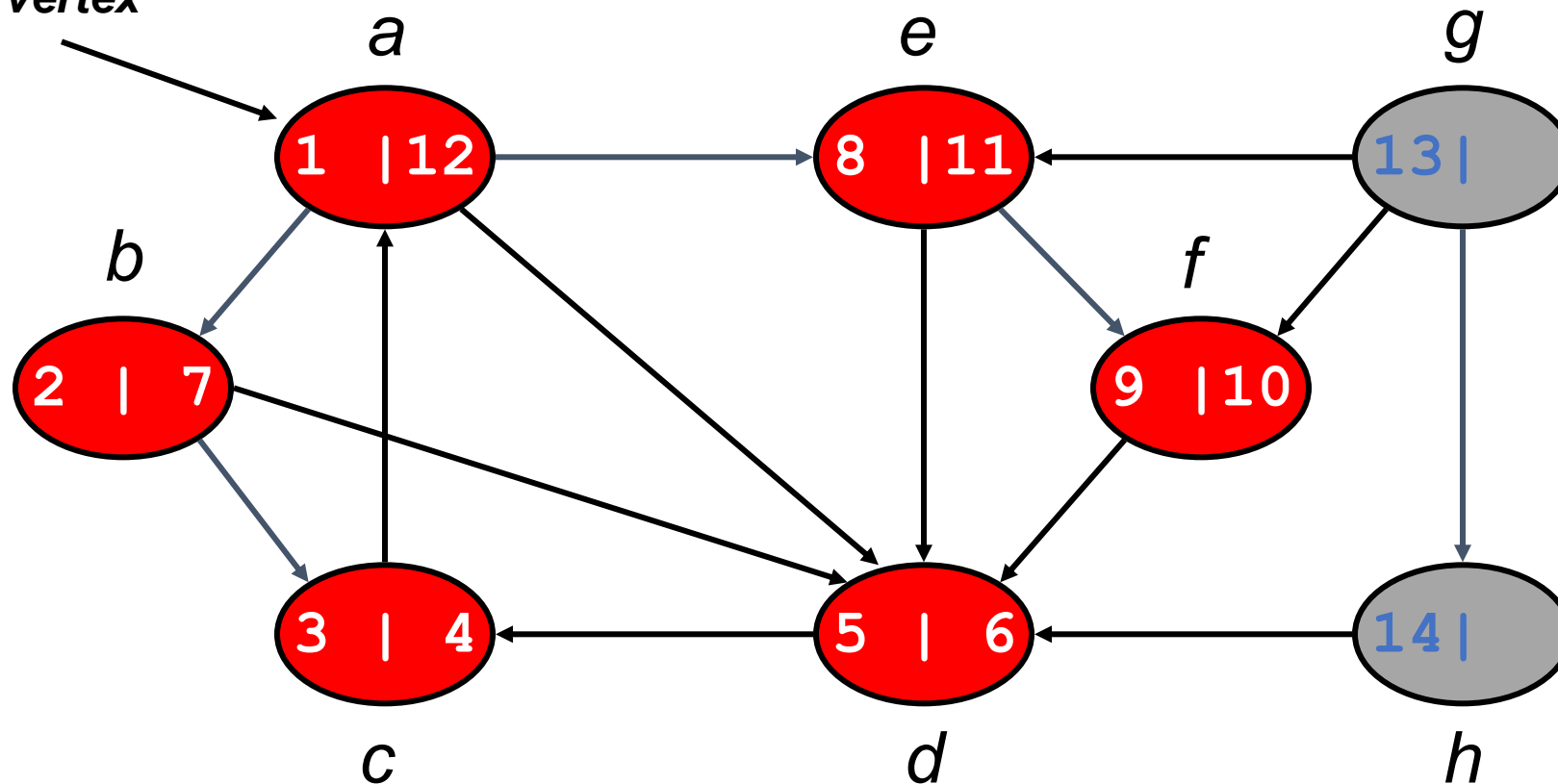
source
vertex



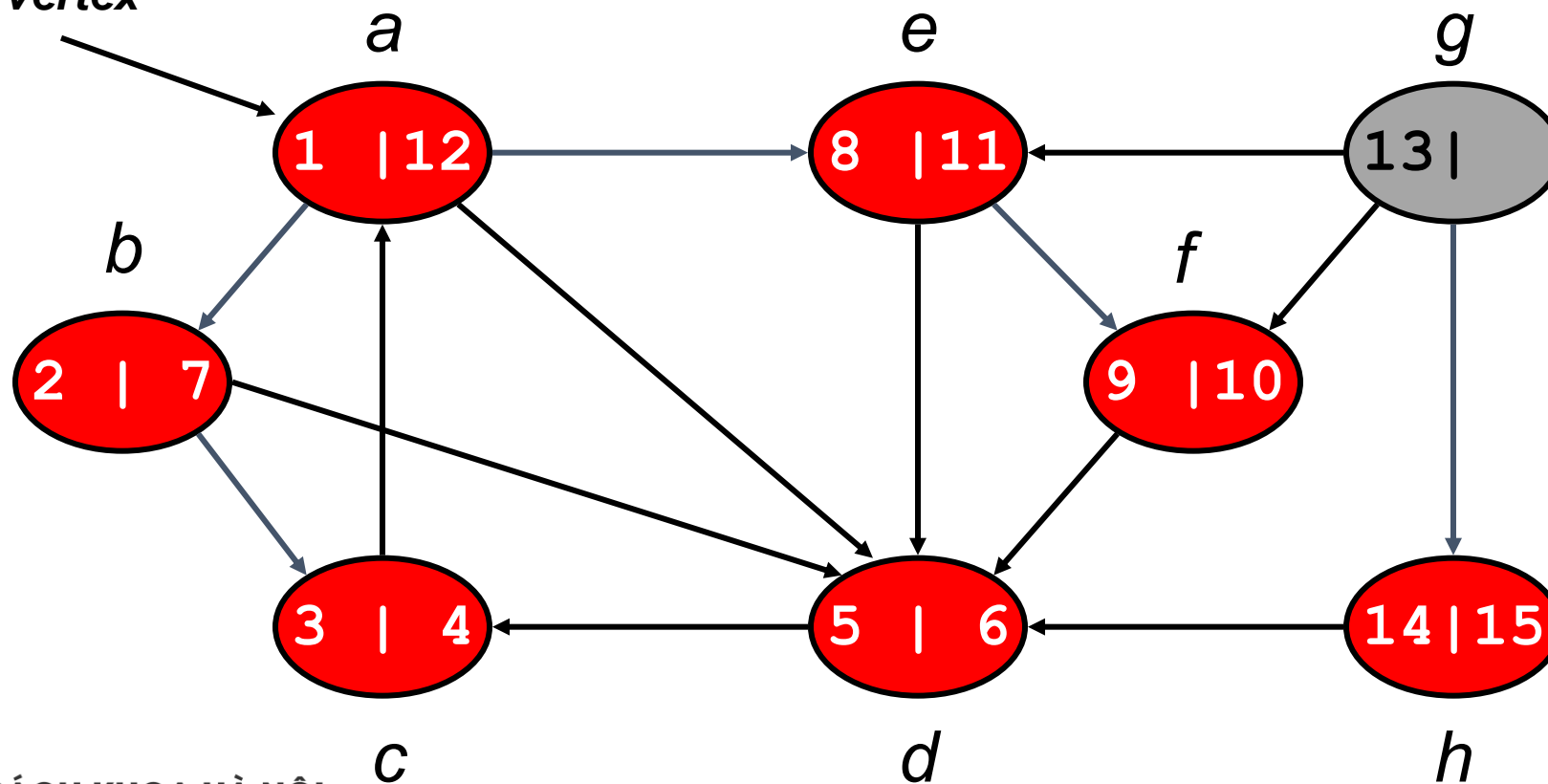
source
vertex



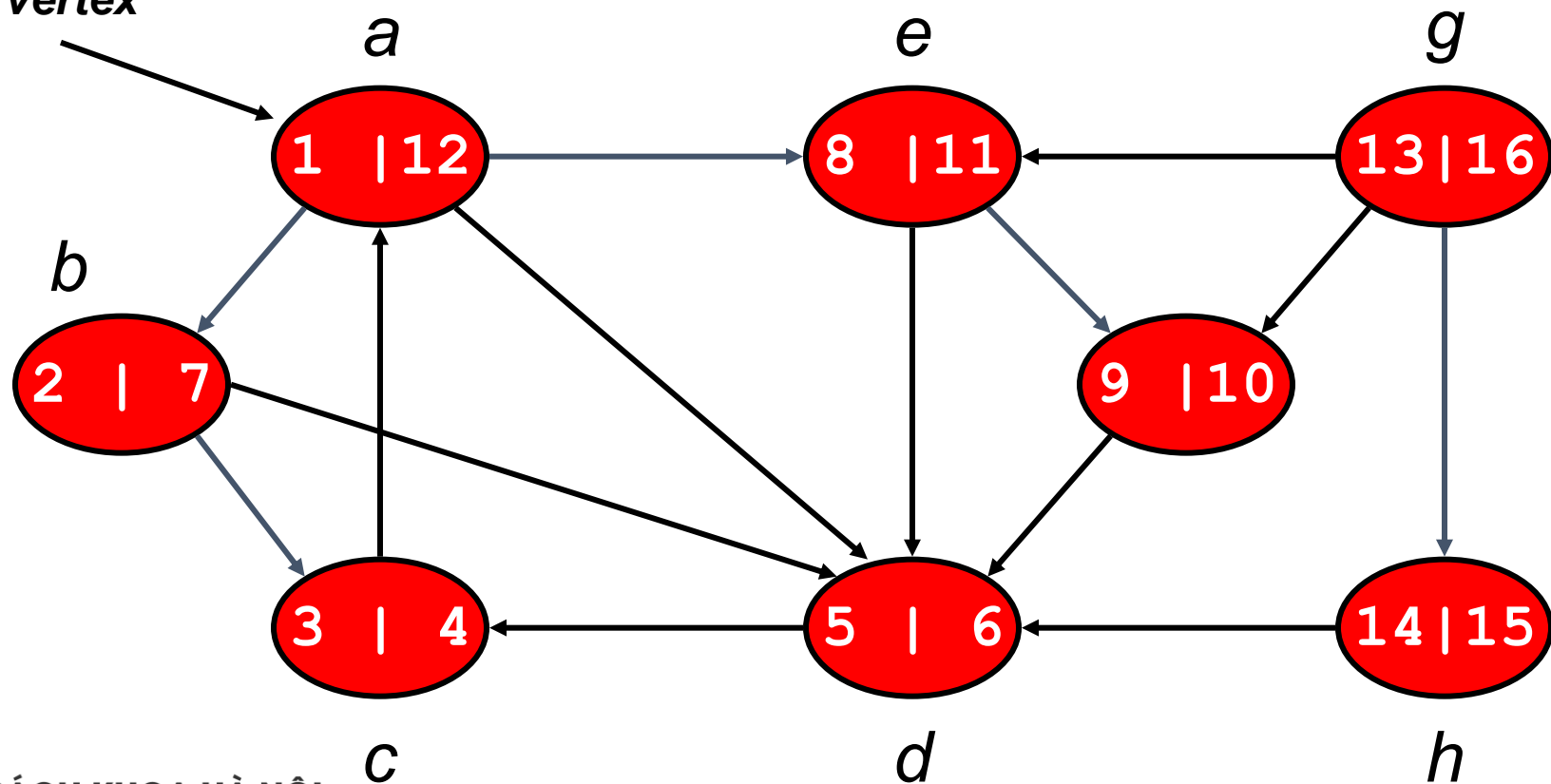
source
vertex



source
vertex



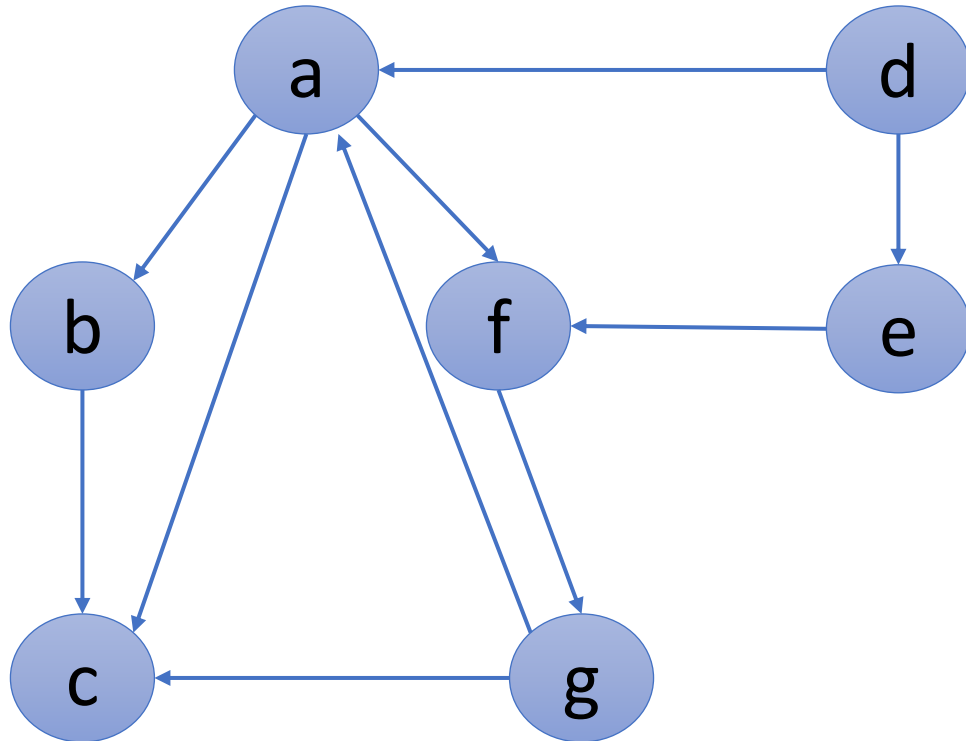
source
vertex



LEMMA OF NESTED INTERVALS

Given directed graph $G = (V, E)$, and arbitrary DFS tree, 2 arbitrary vertices u, v of G . Then

- u is a descendant of v iff $[d[u], f[u]] \subseteq [d[v], f[v]]$
- u is ancestor of v iff $[d[u], f[u]] \supseteq [d[v], f[v]]$
- u and v are not related iff $[d[u], f[u]]$ and $[d[v], f[v]]$ are not intersecting.



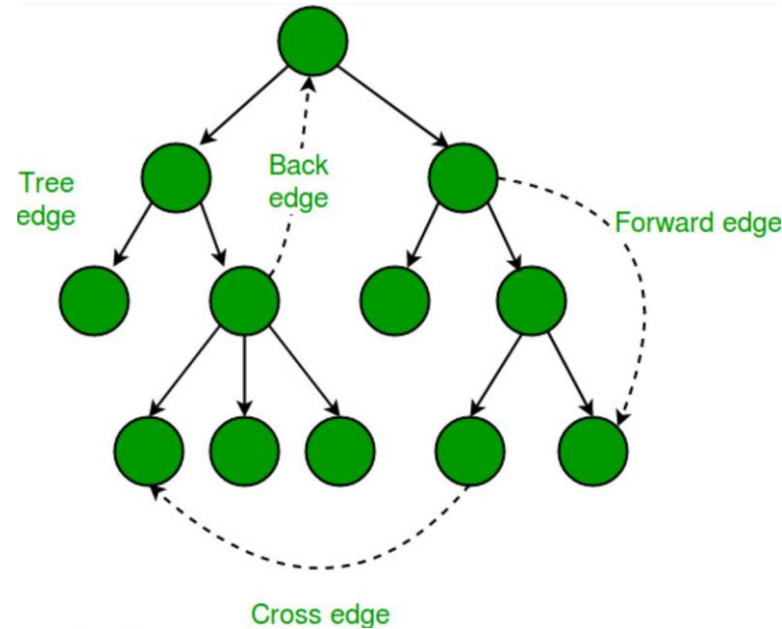
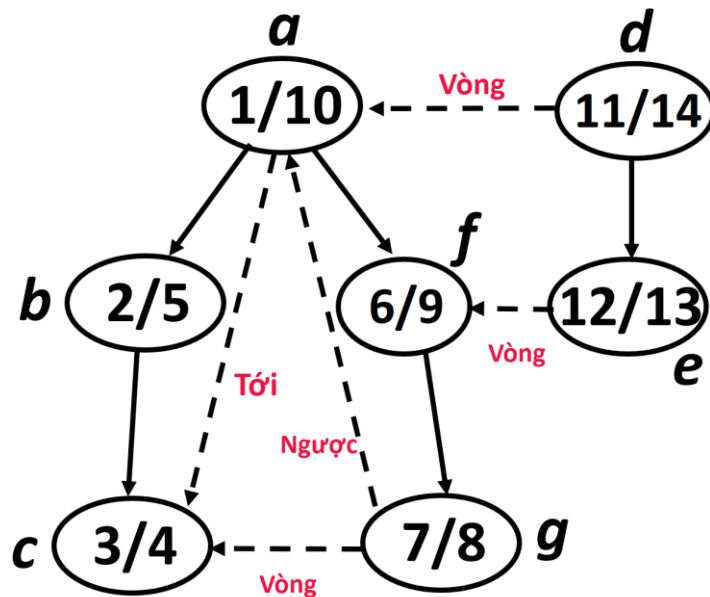
DFS: EDGES CLASSIFICATION

- DFS creates a classification of the edges of given graph:
 - *Tree edge (cạnh cây)*: the edge whereby from a vertex visits a new vertex (cạnh theo đó từ một đỉnh đến thăm đỉnh mới)
 - *Back edge (cạnh ngược)*: going from descendants to ancestors (đi từ con cháu đến tổ tiên)
 - *Forward edge (cạnh tới)*: going from ancestor to descendant (đi từ tổ tiên đến con cháu)
 - *Cross edge (cạnh vòng)*: edge connecting 2 non-related vertices (giữa hai đỉnh không có họ hàng)
- **Note:** there are many applications using tree edges and back edges

LEMMA OF NESTED INTERVALS

Given directed graph $G = (V, E)$, and arbitrary DFS tree, 2 arbitrary vertices u, v of G . Then

- u is a descendant of v iff $[d[u], f[u]] \subseteq [d[v], f[v]]$
- u is ancestor of v iff $[d[u], f[u]] \supseteq [d[v], f[v]]$
- u and v are not related iff $[d[u], f[u]]$ and $[d[v], f[v]]$ are not intersecting.



- Breadth First Search
- Depth First Search
- **Applications**

SOME APPLICATIONS

1. Connectedness of graph
2. Find the path from s to t
3. Cycle detection
4. Check strongly connectedness
5. Graph orientation

THE PROBLEM OF CONNECTIVITY

- **Problem:** Given undirected graph $G = (V, E)$. How many connected components are there in this graph, and each connected component consists of which vertices?
- Answer: Use DFS (BFS) :
 - Each time the function DFS (BFS) is called in the main program, there is one more connected component found in the graph

DFS TO SOLVE THE PROBLEM OF CONNECTIVITY

DFS(*s*)

1. $visited[s] \leftarrow true$ // Thăm đỉnh *s*
2. **for** each $v \in Adj[s]$
3. **if** ($visited[v] == false$)
4. DFS(*v*)

(* Main Program*)

1. **for** $u \in V$
2. $id[u] \leftarrow 0$;
3. $cnt \leftarrow 0$; //cnt - số lượng tplt
4. **for** $u \in V$
5. **if** ($id[u] == 0$) {
6. $cnt \leftarrow cnt + 1$;
7. DFS(*u*) ;
8. }

(* Main Program*)

1. **for** $s \in V$
2. $visited[s] \leftarrow false$
3. **for** $s \in V$
4. **if** ($visited[s] == false$)
5. DFS(*s*)

DFS(*u*)

1. $id[u] \leftarrow cnt$;
2. **for** each $v \in Adj[u]$
3. **if** ($id[v] == 0$)
4. DFS(*v*);

BFS TO SOLVE THE PROBLEM OF CONNECTIVITY

```
BFS(s) {
// Tìm kiếm theo chiều rộng bắt đầu từ đỉnh s
visited[s] ← 1; //đã thăm
Q ← ∅; enqueue(Q,s); // Nạp s vào Q
while (Q ≠ ∅)
{
    u ← dequeue(Q); // Lấy u khỏi Q
    for v ∈ Adj[u]
        if (visited[v] == 0) //chưa thăm
        {
            visited[v] ← 1; //đã thăm
            enqueue(Q,v) // Nạp v vào Q
        }
    }
}

main ()
{
    for s ∈ V // Khởi tạo
        visited[s] ← 0;

    for s ∈ V
        if (visited[s]==0) BFS(s);
}
```



```
BFS(s)
1.   id[s] ← cnt
2.   Q ← ∅; enqueue(Q,s); // Nạp s vào Q
3.   while (Q ≠ ∅)
4.   {
5.       u ← dequeue(Q); // Lấy u khỏi Q
6.       for v ∈ Adj[u]
7.           if (id[v] == 0) //v chưa thăm
8.           {
9.               id[v] ← cnt;
10.          enqueue(Q,v) // Nạp v vào Q
11.          }
12.  }
```

```
(* Main Program*)
1.  for s ∈ V
2.      id[s] ← 0
3.  cnt ← 0 //cnt - số lượng tplt
4.  for s ∈ V
5.      if (id[s] == 0){
6.          cnt ← cnt +1
7.          BFS(s)
8.      }
```

THE PROBLEM OF FINDING THE PATH

The problem of finding the path

- **Input:** Graph $G = (V, E)$ represents by adjacency list, and 2 vertices s, t .
- **Output:** Path from vertex s to vertex t , or confirm there is no path from s to t .

Algorithm: Perform DFS(s) (or BFS(s)).

- If $\text{pred}[t] == \text{NULL}$ then there does not exist the path, otherwise there is the path from s to t and the path is:

$$t \leftarrow \text{pred}[t] \leftarrow \text{pred}[\text{pred}[t]] \leftarrow \dots \leftarrow s$$

DFS TO SOLVE THE PROBLEM OF FINDING THE PATH

DFS(*s*)

```
1.  visited[s] ← true //Thăm đỉnh s
2.  for each v ∈ Adj[s]
3.      if (visited[v] == false) {
4.          truoc[v] ← s
5.          DFS(v)
6.      }
```

(* Main Program*)

```
1.  for u ∈ V {
2.      visited[u] ← false
3.      truoc[u] ← NULL }
4.  DFS(s)
```

BFS TO SOLVE THE PROBLEM OF FINDING THE PATH

(* Main Program*)

```
1. for  $u \in V$  {  
2.     visited[ $u$ ]  $\leftarrow$  false  
3.     truoc[ $u$ ]  $\leftarrow$  NULL }  
4. BFS( $s$ )
```

BFS(s)

```
1.  visited[ $s$ ]  $\leftarrow$  true; //Thăm đỉnh  $s$   
2.  truoc[ $s$ ]  $\leftarrow$  null;  
3.   $Q \leftarrow \emptyset$ ; enqueue( $Q, s$ ); // Nạp  $s$  vào  $Q$   
4.  while ( $Q \neq \emptyset$ )  
5.  {  
6.       $u \leftarrow$  dequeue( $Q$ ); // Lấy  $u$  khỏi  $Q$   
7.      for  $v \in \text{Adj}[u]$   
8.          if (visited[ $v$ ] == false) //chưa thăm  
9.          {  
10.             visited[ $v$ ]  $\leftarrow$  true; //đã thăm  
11.             truoc[ $v$ ]  $\leftarrow$   $u$ ;  
12.             enqueue( $Q, v$ ) //Nạp  $v$  vào  $Q$   
13.          }  
14. }
```

CYCLE DETECTION: USING DFS

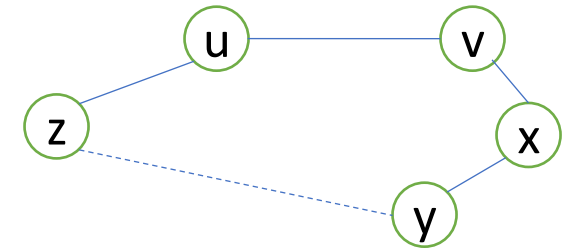
Problem: Given graph $G=(V,E)$. G contains cycle or not?

- **Theorem:** Graph G does not contain cycle if and only if during the DFS execution, we don't not detect the back edge.

Proof:

- $\Rightarrow$ If G does not contain the cycle then there does not exist back edge. Obviously: the existence of back edge (going from descendants to ancestors) entails the existence of cycle.
- $\Leftarrow$ We need to prove: if there does not exist back edge, then G does not contain the cycle. We prove by contrapositive: G has cycle $\Rightarrow \exists$ back edge. Let v be the vertex on the cycle that is the first visited in the DFS execution, and u is the preceding vertex of v on the cycle. When v is visited, the remaining vertices on cycle are all not visited yet. We need to visit all the vertices that are reachable from v before going back v when finishing $\text{DFS}(v)$. Thus, the edge $u \rightarrow v$ is traversed from u to its ancestor v , so (u, v) is back edge.

Therefore, DFS can be used to solve the cycle detection problem.



CYCLE DETECTION: USING DFS

Problem: Given graph $G=(V,E)$. G contains cycle or not?

- **Theorem:** Graph G does not contain cycle if and only if during the DFS execution, we don't not detect the back edge.
 - **The way to detect the existence of back edge:**
 - 1st method: use lemma of nested intervals
 - 2nd method: mark the state for vertices

DFS AND CYCLE DETECTION: METHOD 2

- *How to modify so we can detect the back edge $\rightarrow$ the cycle*

(* Main Program*)

```
1. for  $u \in V$ 
2.     visited[u]  $\leftarrow$  false
3. for  $u \in V$ 
4.     if (visited[u] == false)
5.         DFS(u)
```

DFS(u)

```
1. visited[u]  $\leftarrow$  true //visit u
2. for each  $v \in Adj[u]$ 
3.     if (visited[v] == false)
4.         DFS(v)
```

- Currently, each vertex has 2 states: visited =false or true

DFS AND CYCLE DETECTION: METHOD 2

- Instead of using only **two states: not visited/visited** (visited = 0/1) for each vertex → Need to modify: each vertex has **one of 3 states**:
 - Not visited: visited = 0
 - Visited (but not finish traversal yet): visited = 1
 - Finish traversal: visited = 2
- The state of vertex is changed according to the following rule:
 - Initialize: each vertex v is not visited $visited[v] = 0$
 - Visit v : $visited[v] = 1$ (become visited but not finish traversal yet).
 - When all adjacency vertices of v are visited, vertex v is called as finish traversal $visited[v] = 2$

```
(* Main Program*)
1. for  $u \in V$ 
2.      $visited[u] \leftarrow false$ 
3. for  $u \in V$ 
4.     if ( $visited[u] == false$ )
5.         DFS( $u$ )
```

```
DFS( $u$ )
1.      $visited[u] \leftarrow true$  //visit  $u$ 
2.     for each  $v \in Adj[u]$ 
3.         if ( $visited[v] == false$ )
4.             DFS( $v$ )
```

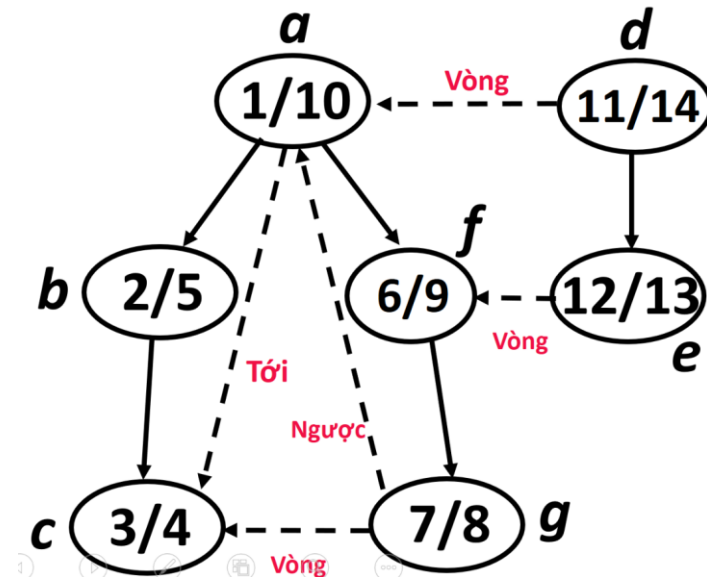
DFS: EDGES CLASSIFICATION

DFS yields edges classification of the graph:

- When we traverse edge $e = (u, v)$ from vertex u , based on the value of $\text{visited}[v]$, we could know the type of edge e :
 - $\text{visited}[v] = 0$: so e is tree edge
 - $\text{visited}[v] = 1$: so e is back edge
 - $\text{visited}[v] = 2$: so e is either forward edge or cross edge

DFS(u)

```
1.  $\text{visited}[u] \leftarrow \text{true}$  //visit  $u$ 
2. for each  $v \in \text{Adj}[u]$ 
3.     if ( $\text{visited}[v] ==$ 
4.         false)
           DFS( $v$ )
```



DFS AND CYCLE DETECTION: METHOD 2

- *How to modify so we can detect the back edge $\rightarrow$ the cycle*

(* Main Program*)

```
1. for  $u \in V$ 
2.   visited[u]  $\leftarrow$  false
3. for  $u \in V$ 
4.   if (visited[u] == false)
5.     DFS(u)
```

DFS(u)

```
1. visited[u]  $\leftarrow$  true //visit u
2. for each  $v \in Adj[u]$ 
3.   if (visited[v] == false)
4.     DFS(v)
```

When we traverse edge $e = (u, v)$ from vertex u , based on the value of $visited[v]$, we could know the type of edge e :

- 1) $visited[v] = 0$: so e is tree edge
- 2) $visited[v] = 1$: so e is back edge

(* Main Program*)

```
1. for  $u \in V$ 
2.   visited[u]  $\leftarrow$  0
3. for  $u \in V$ 
4.   if (visited[u] == 0)
5.     DFS(u)
```

DFS(u)

```
1. visited[u]  $\leftarrow$  1 //visit u
2. for each  $v \in Adj[u]$ 
3.   if (visited[v] == 1)
4.   {
5.     print ("Graph has cycle");
6.     exit();
7.   }
8.   else if(visited[v]==0) DFS(v)
9. visited[u]  $\leftarrow$  2 //u is at state of finish traversal
```

CHECK STRONGLY CONNECTEDNESS OF DIRECTED GRAPH

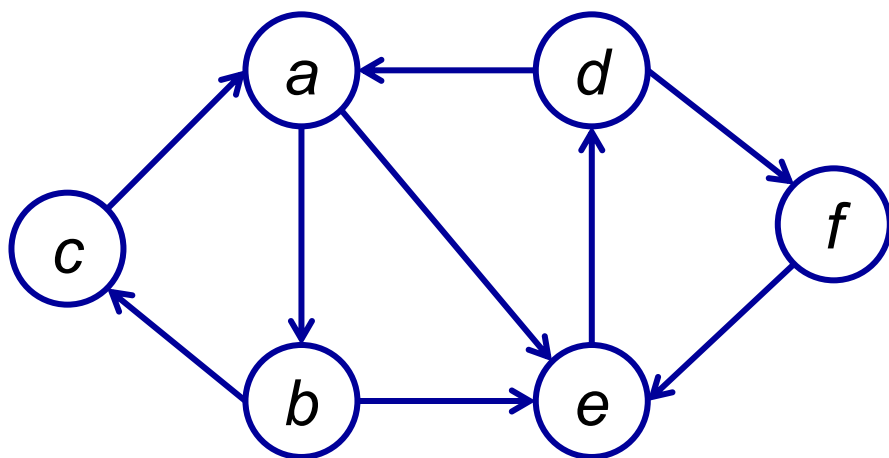
Problem: Given directed graph $G=(V,E)$. Check if the graph G is strongly connected or not?

Proposition: A directed graph $G = (V, E)$ is strongly connected if and only if there always exists a path from a vertex v to all other vertices and always exists a path from all vertices of $V \setminus \{v\}$ to v .

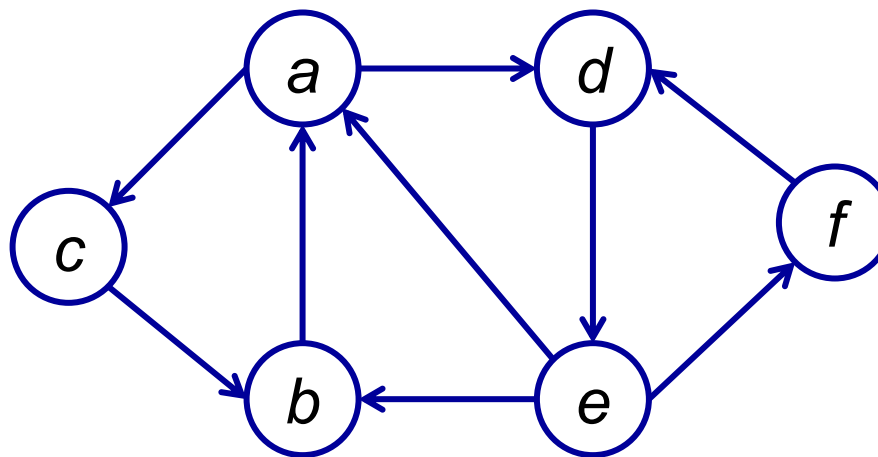
CHECK STRONGLY CONNECTEDNESS OF DIRECTED GRAPH

- Pick an arbitrary vertex $v \in V$.
- Perform DFS(v) on G . If there exists vertex u not visited yet, then G is not strongly connected and the algorithm finishes. Otherwise, the algorithm continues the following step:
 - Perform DFS(v) on $G^T = (V, E^T)$, where E^T is obtained from E by reversing the direction of edges. If exist vertex u not visited, then G is not strongly connected, otherwise G is strongly connected.
- Computation time: $O(|V|+|E|)$

ALGORITHM TO CHECK STRONGLY CONNECTEDNESS OF ~~DIRECTED GRAPH~~



Đồ thị G

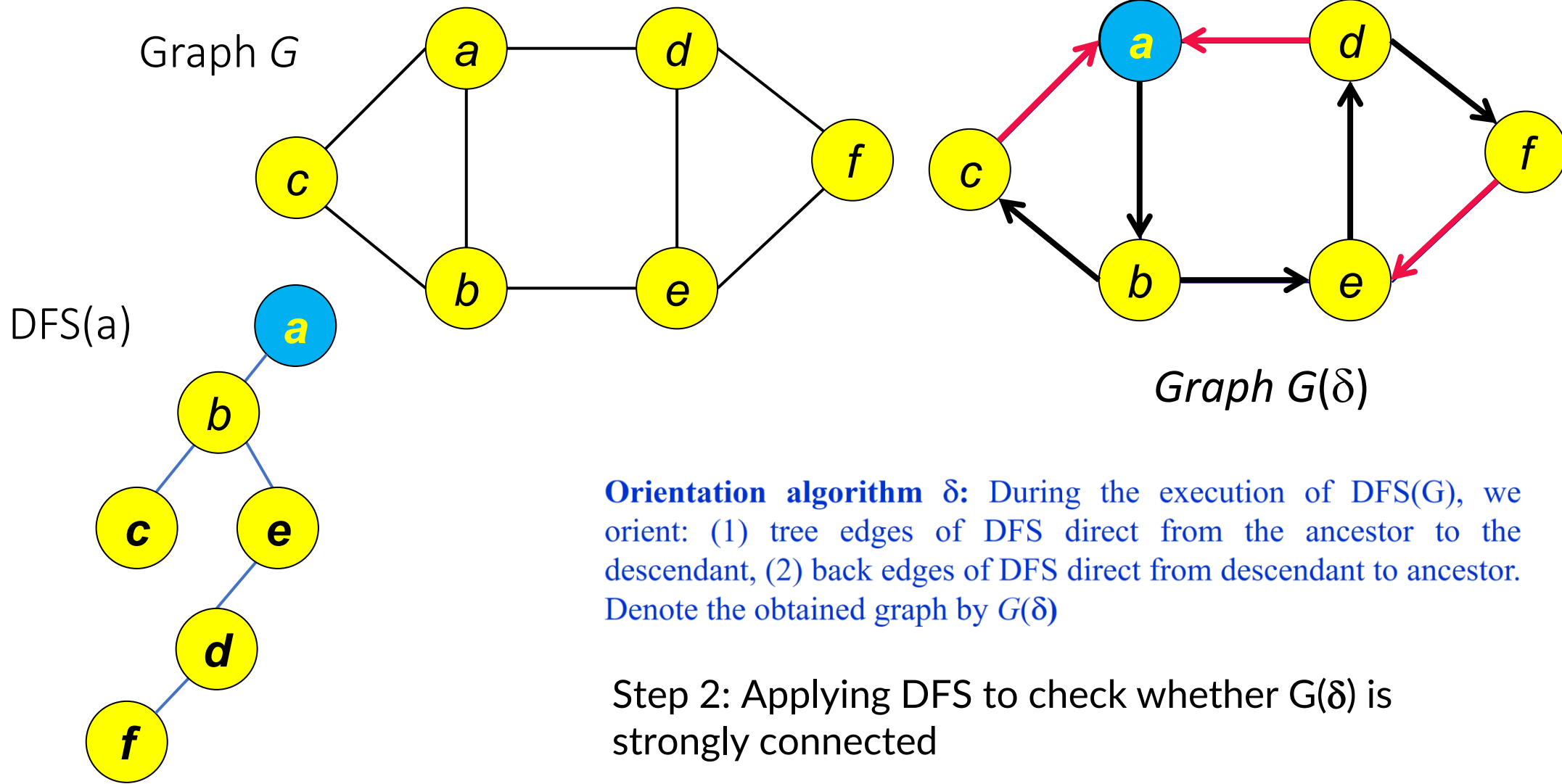


Đồ thị G^T

GRAPH ORIENTATION

- **Problem:** Given undirected connected graph $G = (V, E)$. Find the way to orient its edges such that the obtained directed graph is strongly connected or answer that G is non-directional (G là không định hướng được).
- **Orientation algorithm δ :** During the execution of $\text{DFS}(G)$, we orient: (1) tree edges of DFS direct from the ancestor to the descendant, (2) back edges of DFS direct from descendant to ancestor. Denote the obtained graph by $G(\delta)$
- **Lemma.** G is directional if and only if $G(\delta)$ is strongly connected.

EXAMPLE: GRAPH ORIENTATION



Orientation algorithm δ : During the execution of DFS(G), we orient: (1) tree edges of DFS direct from the ancestor to the descendant, (2) back edges of DFS direct from descendant to ancestor. Denote the obtained graph by $G(\delta)$

Step 2: Applying DFS to check whether $G(\delta)$ is strongly connected

A large graphic on the left side of the slide. It features a dark blue background with a circular pattern of red dots of varying sizes, creating a sense of depth and movement. The word "HUST" is centered within this graphic in a bold, white, sans-serif font.

HUST

THANK YOU !